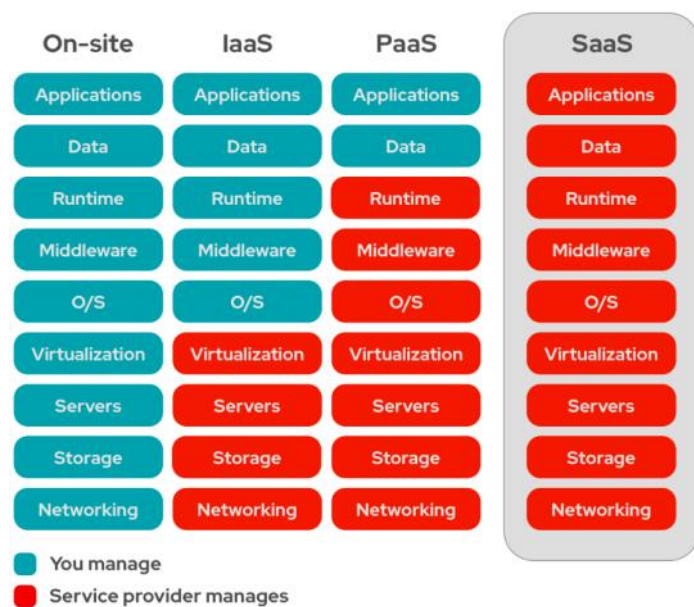


Cloud Computing

lundi 7 juillet 2025 08:36



Principales caractéristiques de l'IaaS :

- **Ressources Réseau** : Accès à des réseaux virtuels, des sous-réseaux, et des services de load balancing.
- **Machines Virtuelles** : Création et gestion de machines virtuelles (VM) pour exécuter des applications et des services.
- **Stockage** : Capacité de stockage flexible et scalable pour les données et les applications.

L'IaaS fournit le niveau de flexibilité et de contrôle le plus élevé parmi les modèles de service cloud. Il est le plus proche des ressources informatiques traditionnelles que l'on trouve dans les départements IT des entreprises, offrant une familiarité aux développeurs et aux administrateurs système.

Exemples de services IaaS chez différents fournisseurs de cloud :

- **Amazon Web Services (AWS)** :
 - **Amazon EC2** : Offre des instances de machines virtuelles avec une flexibilité de configuration des ressources.
 - **Amazon S3** : Service de stockage d'objets scalable et sécurisé.
- **Microsoft Azure** :
 - **Azure Virtual Machines** : Permet de déployer et gérer des machines virtuelles sur une large gamme de configurations.
 - **Azure Blob Storage** : Stockage d'objets pour gérer des données non structurées.
- **Google Cloud Platform (GCP)** :
 - **Google Compute Engine** : Fournit des machines virtuelles personnalisables pour divers besoins de calcul.
 - **Google Cloud Storage** : Stockage d'objets durable et hautement disponible.

Ces services IaaS permettent aux entreprises de louer des ressources IT au lieu d'acheter et de maintenir des équipements physiques, offrant ainsi une scalabilité, une flexibilité et une réduction des coûts significatives.

Platform as a Service (PaaS)

PaaS est un modèle de cloud computing qui fournit une plateforme complète pour le développement, le déploiement et la gestion d'applications sans avoir à gérer l'infrastructure sous-jacente. Il permet aux développeurs de se concentrer sur la création de logiciels tout en laissant le fournisseur de services gérer le matériel, le système d'exploitation, les bases de données et les autres services nécessaires.

Principales caractéristiques du PaaS :

- **Environnements de Développement** : Fournit des outils et des environnements pour le développement d'applications, tels que des IDE (environnements de développement intégrés) et des services de déploiement automatisé.
- **Services de Gestion des Applications** : Inclut des fonctionnalités pour le déploiement, la gestion et la surveillance des applications, facilitant la gestion du cycle de vie des applications.
- **Services Middleware** : Offre des services nécessaires au fonctionnement des applications, tels que des systèmes de gestion de messages et d'autres services d'intégration.

Le PaaS permet aux développeurs de se concentrer sur le code et l'innovation, tout en déléguant la gestion des serveurs, des systèmes d'exploitation et des autres aspects de l'infrastructure au fournisseur de cloud. Cela réduit le besoin de configuration et de maintenance, accélérant ainsi le processus de développement.

Exemples de services PaaS chez différents fournisseurs de cloud :

- **Amazon Web Services (AWS)** :
 - **Amazon Redshift** : Un entrepôt de données entièrement géré qui permet des analyses rapides et évolutives sur de grandes quantités de données.
 - **AWS Glue** : Un service de préparation et de transformation des données qui facilite l'extraction, la transformation et le chargement (ETL) des données.
- **Microsoft Azure** :
 - **Azure Synapse Analytics** : Une solution d'analyse intégrée qui combine des entrepôts de données, des big data et des capacités d'analyse en temps réel.
 - **Azure Data Factory** : Un service de préparation et de gestion des données qui permet d'orchestrer et d'automatiser les flux de travail ETL.
- **Google Cloud Platform (GCP)** :
 - **BigQuery** : Un entrepôt de données serverless et entièrement géré pour des analyses rapides et interactives sur de grandes quantités de données.
 - **Google Cloud Dataflow** : Un service de traitement de données en temps réel et par lots qui facilite l'exécution de pipelines de données.

Les services PaaS offrent une approche simplifiée et efficace pour le développement et la gestion d'applications, en intégrant des outils et des services prêts à l'emploi tout en simplifiant la gestion de l'infrastructure.

Software as a Service (SaaS)

SaaS est un modèle de cloud computing où les applications sont fournies en tant que services via Internet. Les utilisateurs accèdent aux applications directement à travers un navigateur web, sans avoir besoin de les installer ou de les gérer localement. Le fournisseur de SaaS gère l'infrastructure, les serveurs, les bases de données, ainsi que les mises à jour et la maintenance des applications.

Principales caractéristiques du SaaS :

- **Accessibilité** : Les applications SaaS sont accessibles depuis n'importe quel appareil connecté à Internet, ce qui facilite l'accès aux outils et aux données à tout moment et de n'importe où.
- **Gestion Simplifiée** : Le fournisseur gère la maintenance, les mises à jour et la sécurité des applications, permettant aux utilisateurs de se concentrer sur l'utilisation des applications sans se soucier des aspects techniques sous-jacents.
- **Modèle d'Abonnement** : Les services SaaS sont généralement proposés sous forme d'abonnement, avec des options de paiement récurrentes, ce qui permet aux utilisateurs de payer pour ce qu'ils utilisent sans investissement initial important.

Le SaaS offre une solution clé en main pour l'accès aux applications nécessaires à la gestion et à l'analyse des données, sans les tracas associés à l'installation et à la maintenance des logiciels.

Exemples de services SaaS orientés données chez différents fournisseurs de cloud :

- **Amazon Web Services (AWS) :**
 - **Amazon QuickSight :** Un service de visualisation de données et de business intelligence qui permet de créer des tableaux de bord interactifs et des rapports d'analyse sans nécessiter de gestion d'infrastructure.
 - **Amazon Chime :** Un service de communication et de collaboration en ligne qui intègre des fonctions de messagerie, de réunions et de gestion des tâches.
- **Microsoft Azure :**
 - **Power BI :** Un service de business intelligence qui fournit des outils pour la visualisation des données et l'analyse interactive, facilitant la création de rapports et de tableaux de bord.
 - **Office 365 :** Un ensemble d'applications bureautiques basées sur le cloud, incluant des services de messagerie, de collaboration et de gestion de documents.
- **Google Cloud Platform (GCP) :**
 - **Google Data Studio :** Un service de visualisation des données qui permet de créer des rapports et des tableaux de bord interactifs en se connectant à diverses sources de données.
 - **Google Workspace :** Une suite d'outils de productivité et de collaboration en ligne, comprenant Gmail, Google Drive, Google Docs, et d'autres applications pour la gestion et le partage de documents.

Les services SaaS offrent des solutions complètes pour les besoins fonctionnels et analytiques des entreprises, en intégrant des applications prêtes à l'emploi tout en éliminant les complexités liées à l'infrastructure et à la maintenance.

Résumé des Types de Cloud

Voici un aperçu des principaux types de déploiement en cloud computing:

Type de Cloud	Définition	Caractéristiques	Avantages
Cloud Public	Ressources de cloud computing fournies par un tiers (AWS, Azure...) et partagées entre plusieurs clients.	- Accès partagé aux ressources, Scalabilité élevée, Coût basé sur l'usage	- Coût réduit, Scalabilité élevée, Maintenance gérée par le fournisseur
Cloud Privé	Infrastructure de cloud dédiée à une seule organisation, hébergée en interne.	- Accès exclusif aux ressources, Contrôle total sur l'infrastructure, Coût généralement plus élevé	- Sécurité accrue, Personnalisation selon les besoins, Conformité réglementaire améliorée
Cloud Hybride	Combinaison de cloud public et privé permettant l'intégration et la circulation des données entre les deux.	- Flexibilité dans le déploiement, Intégration entre cloud public et privé, Optimisation des coûts	- Flexibilité dans le déploiement des charges de travail, Scalabilité en utilisant le cloud public, Sécurité avec le cloud privé pour les données sensibles
Multi-Cloud	Utilisation de plusieurs services de cloud computing de différents fournisseurs sans nécessité d'intégration entre eux.	- Diversité de fournisseurs, Répartition des risques, Optimisation des services	- Réduction du risque de dépendance au fournisseur, Optimisation des coûts et des services, Flexibilité accrue

Importance des Différents Types de Déploiement

- **Cloud Public :** Idéal pour les entreprises qui cherchent à réduire les coûts et à bénéficier d'une haute scalabilité sans gérer l'infrastructure sous-jacente. C'est une option économique et flexible pour les applications et services qui ne nécessitent pas de sécurité et de conformité strictes.
- **Cloud Privé :** Recommandé pour les organisations qui nécessitent un contrôle total sur leur infrastructure et des niveaux élevés de sécurité et de conformité. Bien que plus coûteux, il permet une personnalisation complète et une protection accrue des données sensibles.
- **Cloud Hybride :** Permet de combiner les avantages des clouds public et privé, offrant ainsi une flexibilité pour déployer des applications sensibles sur un cloud privé tout en utilisant le cloud public pour les charges de travail plus dynamiques. Cela permet une gestion optimisée des coûts et des ressources.
- **Multi-Cloud :** Approche qui utilise plusieurs fournisseurs de cloud pour éviter la dépendance à un seul fournisseur et pour tirer parti des meilleures offres disponibles. Cela permet également une meilleure résilience et optimisation des services en fonction des besoins spécifiques des applications.

1.2 Cloud Computing : Le Cloud et la Data

Introduction

Dans un monde où les données sont devenues le nouveau pétrole, leur gestion, analyse et exploitation sont au cœur des stratégies des entreprises modernes. Les volumes de données générés et collectés ne cessent de croître, rendant les approches traditionnelles de gestion de données rapidement obsolètes. C'est ici que le cloud computing s'impose comme une solution incontournable.

Le cloud, avec sa capacité à offrir une infrastructure flexible, scalable et accessible, transforme radicalement la manière dont les entreprises abordent la gestion de leurs données. Il permet non seulement de stocker et de protéger des volumes massifs de données, mais aussi de les analyser en temps réel, ouvrant la voie à une prise de décision plus rapide et mieux informée. Cette révolution du cloud dans un contexte data-driven permet aux organisations de maximiser la valeur de leurs données tout en réduisant les coûts et en augmentant leur agilité.

Ce guide explore les enjeux et avantages du cloud pour la gestion et l'analyse des données, ainsi que les outils et technologies qui permettent aux entreprises de rester compétitives dans cette ère dominée par les données.

Cloud et Data



Le Rôle du Cloud dans un Environnement Data-Driven

Dans un monde où les données sont au cœur de la prise de décision, le cloud joue un rôle crucial en offrant des solutions flexibles et scalables pour gérer, stocker et analyser ces données. Les environnements data-driven nécessitent une infrastructure capable de s'adapter rapidement aux besoins changeants et de traiter des volumes de données toujours croissants. Le cloud permet d'accéder à des ressources illimitées, de bénéficier de puissants outils d'analyse et de collaborer en temps réel, tout en réduisant les coûts et les contraintes liées à la gestion d'une infrastructure locale. En s'appuyant sur le cloud, les entreprises peuvent ainsi se concentrer sur l'extraction de valeur à partir de leurs données, plutôt que sur la maintenance de leur infrastructure.

Les Défis de la Gestion des Données à Grande Échelle

La gestion des données à grande échelle pose des défis complexes pour les entreprises modernes. À mesure que les volumes de données augmentent, les solutions traditionnelles deviennent inefficaces, entraînant des problèmes de stockage, de traitement et d'analyse. La nécessité de traiter des données provenant de multiples sources en temps réel, tout en assurant leur sécurité et leur conformité, ajoute une couche supplémentaire de complexité. De plus, la scalabilité des infrastructures devient un enjeu majeur pour éviter les goulots d'étranglement et garantir des performances optimales. Le cloud, avec ses capacités d'expansion rapide et ses services spécialisés, répond à ces défis en offrant des solutions robustes et flexibles pour gérer et exploiter les données à grande échelle, tout en réduisant les coûts et en améliorant l'efficacité opérationnelle.

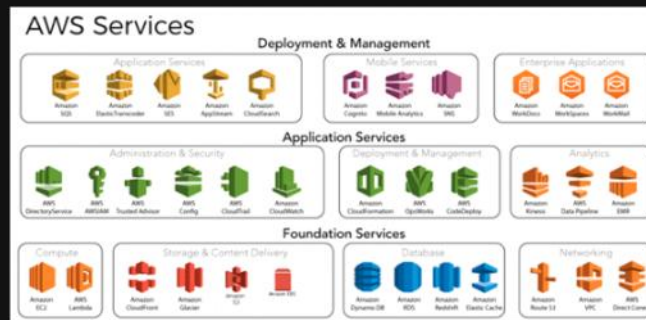
Avantages du Cloud pour la Gestion de la Data

Scalabilité et Flexibilité

La **Scalabilité** dans le cloud se distingue par sa capacité à s'ajuster en fonction des variations de la demande. Cela inclut non seulement le scale up, qui consiste à ajouter des ressources lorsque la demande augmente, mais aussi le scale down, permettant de réduire les ressources lorsque la demande diminue. Ce principe d'élasticité du cloud garantit que les ressources sont ajustées de manière fluide et efficace. Par exemple, si une application connaît un pic de trafic, le cloud permet d'augmenter rapidement les capacités de traitement et de stockage. À l'inverse, lorsque la demande baisse, les ressources peuvent être réduites pour diminuer les coûts, optimisant ainsi l'investissement et les dépenses.

La Flexibilité offre la possibilité d'adapter les ressources et les services selon les besoins spécifiques de chaque entreprise. Le cloud permet de choisir et de combiner divers outils et services en fonction des exigences particulières en matière d'analyse des données, de stockage ou de traitement. Cette adaptabilité facilite non seulement la réponse aux besoins actuels, mais aussi la réactivité face aux changements de stratégie ou aux évolutions technologiques.

Illustration de différents services disponibles chez AWS



Réduction des Coûts et Efficacité Opérationnelle

La **Réduction des coûts** est l'un des avantages majeurs du cloud. Contrairement aux infrastructures traditionnelles, où les entreprises doivent investir massivement dans du matériel, des licences logicielles, et des coûts de maintenance, le cloud fonctionne sur un modèle de paiement à l'usage. Cela signifie que les entreprises ne paient que pour les ressources qu'elles utilisent, évitant ainsi les coûts initiaux élevés et les dépenses liées à la surcapacité. De plus, grâce au scale down, elles peuvent réduire les ressources en période de faible demande, diminuant encore davantage les coûts opérationnels.

Le cloud permet une transition de coûts d'investissements initiaux élevés vers des coûts opérationnels plus prévisibles

L'**Efficacité opérationnelle** est également améliorée grâce à la gestion automatisée et centralisée des ressources offertes par les fournisseurs de cloud. Les tâches qui nécessitaient autrefois une intervention manuelle, telles que la maintenance des serveurs, la gestion des mises à jour ou l'optimisation des performances, sont désormais automatisées, permettant aux équipes IT de se concentrer sur des tâches à plus forte valeur ajoutée. De plus, l'accès à des outils avancés d'analyse, de stockage et de traitement des données permet de maximiser l'utilisation des ressources tout en minimisant les inefficacités.

Sécurité et Conformité

La **sécurité des données** est une préoccupation majeure pour toute organisation, et le cloud peut offrir des solutions robustes pour répondre à ces enjeux. Les principaux fournisseurs de cloud investissent massivement dans des technologies de sécurité avancées, souvent difficiles à déployer en interne. Ces technologies incluent des mécanismes de chiffrement des données en transit et au repos, des systèmes de détection des intrusions, des pare-feu avancés, ainsi que des outils d'audit et de surveillance continue.

En ce qui concerne la **conformité**, les fournisseurs de cloud proposent une large gamme de certifications et de normes de conformité, telles que GDPR, HIPAA, ISO 27001, et bien d'autres. Ils mettent également à disposition des outils pour aider les entreprises à respecter ces réglementations en facilitant la gestion des données sensibles et en offrant des options de géolocalisation pour les données, permettant ainsi de s'assurer que les informations restent dans des juridictions spécifiques.

Sécurité et Conformité

La **sécurité** est au cœur d'Amazon S3, avec des options de chiffrement des données au repos et en transit. S3 permet également de définir des **politiques** de contrôle d'accès précises, afin de gérer les droits des utilisateurs au niveau du bucket ou de l'objet. De plus, S3 est conforme à de nombreuses normes de sécurité et de conformité, telles que HIPAA*, PCI-DSS*, RGPD et SOC*, garantissant que les données sensibles sont protégées conformément aux exigences réglementaires.

■ Informations sur les Normes de Conformité :

HIPAA (Health Insurance Portability and Accountability Act) : Une loi américaine qui impose des normes de protection des informations de santé des individus, garantissant leur confidentialité, sécurité, et intégrité. Elle est essentielle pour toute organisation gérant des données de santé aux États-Unis.

SOC (System and Organization Controls) : Un cadre de conformité qui inclut des rapports SOC 1, SOC 2, et SOC 3, évaluant les contrôles de sécurité, de disponibilité, de traitement de l'intégrité des systèmes, et de confidentialité dans une organisation.

PCI-DSS (Payment Card Industry Data Security Standard) : Une norme de sécurité globale pour toute entreprise qui accepte, traite, stocke ou transmet des informations de carte de crédit, garantissant un environnement sécurisé pour prévenir les fraudes et les violations de données.

Rappel RGPD (Règlement Général sur la Protection des Données) : Un règlement européen qui renforce et unifie la protection des données pour tous les individus au sein de l'Union européenne. Il impose des obligations strictes sur la collecte, le stockage et le traitement des données personnelles, avec des sanctions sévères en cas de non-conformité.

Installer boto3

Avec l'environnement virtuel activé, installez boto3 via pip :

```
1 pip install boto3
```

2. Créez un script

Une fois que boto3 est installé et que vos identifiants AWS sont configurés, vous pouvez créer un script python dans votre dossier, appelez le comme vous voulez par exemple : `mon_script_boto.py`

Boto3 et s3

Créer un bucket s3 avec boto3

Créer un bucket S3 est la première étape pour gérer vos fichiers dans S3. Voici comment le faire avec `boto3`.

```
1 import boto3
2
3 # Créer une session boto3
4 session = boto3.Session()
5
6 # Utiliser le client S3
7 s3 = session.client('s3', region_name='eu-west-3')
8
9 # Nommer votre bucket (le nom doit être unique globalement)
10 bucket_name = 'mon-nouveau-bucket-exemple'
11
12 # Créer le bucket
13 s3.create_bucket(Bucket=bucket_name,
14                  CreateBucketConfiguration={
15                      'LocationConstraint': 'eu-west-3' # Spécification de la région
16                  })
17
18 print(f"Bucket '{bucket_name}' créé avec succès.")
```

Explication du code :

- `session = boto3.Session()`: Crée une session avec vos identifiants AWS configurés.
- `s3 = session.client('s3')`: Initialise un client pour interagir avec S3.
- `bucket_name = 'mon-nouveau-bucket-exemple'`: Spécifie le nom du bucket que vous souhaitez créer.
- `s3.create_bucket(Bucket=bucket_name)`: Crée le bucket S3 avec le nom spécifié.

Lister les buckets S3

Pour lister tous les buckets S3 associés à votre compte AWS avec boto3 :

```
1 import boto3
2
3 # Créer une session boto3
4 session = boto3.Session()
5
6 # Utiliser le client S3
7 s3 = session.client('s3')
8
9 # Lister les buckets
10 response = s3.list_buckets()
11
12 print("Liste des buckets S3 :")
13 for bucket in response['Buckets']:
14     print(f" - {bucket['Name']}")
15
```

Explication du code :

- `response = s3.list_buckets()`: Récupère la liste de tous les buckets S3 de votre compte.
- `for bucket in response['Buckets']`: Boucle sur chaque bucket trouvé dans la réponse.
- `print(f" - {bucket['Name']}")`: Affiche le nom de chaque bucket.

Uploader un fichier dans un bucket S3

Uploader des fichiers dans un bucket S3 est une tâche courante on a déjà vu comment faire cette tâche avec AWS console management et AWS CLI on va voir comment le faire avec boto3

```
1 import boto3
2
3 # Créer une session boto3
4 session = boto3.Session()
5
6 # Utiliser le client S3
7 s3 = session.client('s3')
8
9 # Nom du fichier local et du bucket
10 file_name = 'chemin/vers/votre/fichier.txt'
11 bucket_name = 'mon-nouveau-bucket-exemple'
12
13 # Uploader le fichier
14 s3.upload_file(file_name, bucket_name, 'fichier_s3.txt')
15
16 print(f"Fichier '{file_name}' uploadé avec succès dans le bucket '{bucket_name}'.")
```

Explication du code :

- `file_name = 'chemin/vers/votre/fichier.txt'`: Spécifie le chemin vers le fichier que vous souhaitez uploader.
- `s3.upload_file(file_name, bucket_name, 'fichier_s3.txt')`: Uploader le fichier local dans le bucket S3 spécifié avec le nom donné dans S3 (fichier_s3.txt).

Télécharger un fichier depuis un bucket S3

Pour télécharger un fichier depuis un bucket S3 vers votre machine locale :

```
1 import boto3
2
3 # Créer une session boto3
4 session = boto3.Session()
5
6 # Utiliser le client S3
7 s3 = session.client('s3')
8
9 # Nom du fichier S3 et local
10 bucket_name = 'mon-nouveau-bucket-exemple'
11 file_name = 'fichier_s3.txt'
12 local_file_name = 'fichier_téléchargé.txt'
13
14 # Télécharger le fichier
15 s3.download_file(bucket_name, file_name, local_file_name)
16
17 print(f"Fichier '{file_name}' téléchargé avec succès depuis le bucket '{bucket_name}'.")
```

Explication du code :

`file_name = 'fichier_s3.txt'`: Nom du fichier dans S3 que vous souhaitez télécharger.
`local_file_name = 'fichier_téléchargé.txt'`: Nom du fichier sur votre machine après le téléchargement.
`s3.download_file(bucket_name, file_name, local_file_name)`: Télécharge le fichier depuis S3 vers votre machine locale.

Supprimer un fichier d'un bucket S3

Si vous souhaitez supprimer un fichier d'un bucket S3, ou plusieurs fichiers :

```
1 import boto3
2
3 # Créer une session boto3
4 session = boto3.Session()
5
6 # Utiliser le client S3
7 s3 = session.client('s3')
8
9 # Nom du fichier à supprimer et du bucket
10 bucket_name = 'mon-nouveau-bucket-exemple'
11 file_name = 'fichier_s3.txt'
12
13 # Supprimer le fichier
14 s3.delete_object(Bucket=bucket_name, Key=file_name)
15
16 print(f"Fichier '{file_name}' supprimé avec succès du bucket '{bucket_name}'.")
```

Explication du code :

- `s3.delete_object(Bucket=bucket_name, Key=file_name)`: Supprime l'objet spécifié dans le bucket S3.
- Le paramètre `Key` représente le nom complet (ou chemin) de l'objet que vous voulez supprimer dans le bucket S3. En d'autres termes, c'est l'identifiant unique de l'objet au sein du bucket.
- La valeur de `Key` est une chaîne de caractères qui correspond au nom de l'objet dans le bucket S3. Cela peut être simplement un nom de fichier ou un chemin qui inclut des répertoires virtuels (comme dossier1/dossier2/fichier.txt).

Supprimer un bucket S3

Pour supprimer un bucket S3, il faut s'assurer qu'il soit vide dans un premier temps sinon ça ne fonctionnera pas, assurez vous donc de bien avoir vidé le bucket avant d'essayer de le supprimer.

```
1 import boto3
2
3 # Créer une session boto3
4 session = boto3.Session()
5
6 # Utiliser le client S3
7 s3 = session.client('s3')
8
9 # Nom du bucket à supprimer
10 bucket_name = 'mon-nouveau-bucket-exemple'
11
12 # Supprimer le bucket
13 s3.delete_bucket(Bucket=bucket_name)
14
15 print(f"Bucket '{bucket_name}' supprimé avec succès.")
```

Explication du code :

`s3.delete_bucket(Bucket=bucket_name)` : Supprime le bucket S3 spécifié. Le bucket doit être vide avant la suppression.

Amazon Redshift

Introduction

Dans cette quête, vous allez découvrir et prendre en main Amazon Redshift, un puissant service de **DataWarehousing** cloud proposé par AWS. Vous commencerez par une présentation générale d'Amazon Redshift, où vous apprendrez ce qu'est ce service, comment il fonctionne, et pourquoi il est avantageux pour des tâches d'analyse de données à grande échelle.

Ensuite, vous passerez à la pratique en prenant en main Amazon Redshift Serverless. Vous verrez comment créer une instance Redshift Serverless, puis comment sécuriser vos accès en paramétrant les identifiants via AWS Secrets Manager. Vous apprendrez également à configurer un Security Group pour permettre un accès TCP sécurisé à votre instance.

Pour terminer, vous testerez votre configuration en chargeant et en manipulant des données avec SQLAlchemy. Cela vous permettra de valider le bon fonctionnement de votre environnement Redshift.

En conclusion, cette quête vous fournira les bases nécessaires pour comprendre, configurer, et utiliser Amazon Redshift Serverless dans vos projets de traitement de données.

Présentation d'Amazon Redshift

Qu'est-ce qu'Amazon Redshift ?

Amazon Redshift est un service d'entrepôt de données (data warehouse) entièrement géré à l'échelle du pétaoctet, conçu pour le stockage et l'analyse de grandes quantités de données. Basé sur PostgreSQL, Redshift est spécifiquement optimisé pour les charges de travail OLAP (Online Analytical Processing), ce qui en fait un choix idéal pour les entreprises cherchant à effectuer des analyses complexes sur de vastes ensembles de données.

Redshift offre des performances jusqu'à 10 fois supérieures à celles d'autres entrepôts de données grâce à son architecture Massively Parallel Processing (MPP). Ce modèle d'exécution en parallèle permet de distribuer les charges de travail à travers de multiples nœuds de calcul, augmentant ainsi l'efficacité et la vitesse des requêtes.

Le service utilise un stockage en colonnes pour optimiser les requêtes analytiques, ce qui réduit la quantité de données à lire lors des requêtes complexes, améliorant ainsi les temps de réponse. Redshift adopte également un modèle de tarification "pay as you go", permettant aux utilisateurs de ne payer que pour les ressources qu'ils consomment, ce qui en fait une solution flexible et économique.

Amazon Redshift peut être intégré avec des outils de Business Intelligence (BI) tels qu'Amazon QuickSight ou Tableau, facilitant ainsi la visualisation et l'exploration des données stockées. En résumé, Amazon Redshift est une solution puissante et flexible pour les entreprises qui ont besoin de stocker, analyser et tirer des insights de grandes quantités de données, tout en bénéficiant d'une gestion simplifiée et d'une performance optimale.

▲ Rappel : Qu'est ce que le stockage en colonne ?

Le stockage en colonne (ou columnar storage) est une méthode d'organisation des données dans une base de données où les valeurs des colonnes sont stockées ensemble, plutôt que les valeurs des lignes.

Avantage principal : Cette méthode est particulièrement efficace pour les requêtes analytiques, car elle permet de lire uniquement les colonnes nécessaires, réduisant ainsi la quantité de données scannées et améliorant considérablement les performances des requêtes.

Comment fonctionne Amazon Redshift ?

Dans cette partie vous allez voir comment fonctionne Redshift une fois que vous lui envoyez une requête. Les éléments clés de ces exécutions sont : le Leader Node, les Compute Nodes et les Slices. Nous allons prendre une requête simple pour bien comprendre le fonctionnement de Redshift.

Imaginons que tu aies une base de données dans Amazon Redshift avec une table appelée **ventes**, et que tu veuilles exécuter la requête SQL suivante :

```
1 SELECT produit, SUM(montant)
2 FROM ventes
3 WHERE date_vente BETWEEN '2024-01-01' AND '2024-12-31'
4 GROUP BY produit;
```

Voici comment Redshift va traiter ta demande :

1. Le Leader Node (Nœud de Coordination) :

- **Réception de la Requête** : Le Leader Node reçoit ta requête SQL.
- **Planification** : Il analyse la requête et crée un plan d'exécution. Ce plan définit comment les données doivent être filtrées, agrégées et regroupées.
- **Distribution des Tâches** : Le Leader Node divise le travail en plusieurs tâches et les distribue aux Compute Nodes.

2. Compute Nodes (Nœuds de Calcul) :

- **Filtrage des Données** : Chaque Compute Node récupère une partie des données de la table **ventes**. Les Compute Nodes utilisent des Slices pour gérer des portions spécifiques des données.
- **Application des Filtres** : Chaque Compute Node filtre les données pour ne conserver que celles où **date_vente** est entre '2024-01-01' et '2024-12-31'.
- **Calcul des Agrégats** : Les Compute Nodes calculent la somme des montants pour chaque produit dans leurs portions de données respectives.

3. Combinaison des Résultats :

- **Agrégation des Résultats** : Les résultats partiels de chaque Compute Node sont envoyés au Leader Node.
- **Consolidation des Données** : Le Leader Node combine les résultats partiels des Compute Nodes pour obtenir le résultat final de la requête : la somme des montants par produit.

4. Envoi des Résultats :

- **Retour des Résultats** : Enfin, le Leader Node envoie le résultat consolidé à l'utilisateur qui a fait la requête.

Prise en main d'Amazon Redshift Serverless

Création d'une instance Redshift Serverless

1. Connexion à la console AWS

- Rendez-vous sur **AWS Management Console**.
- Connectez-vous à votre compte AWS avec vos identifiants.

2. Accès à Redshift Serverless via la barre de recherche

- Dans la barre de recherche en haut de la page, tapez **Redshift**.
- Sélectionnez **Amazon Redshift** dans les résultats pour accéder à la page de gestion d'Amazon Redshift.

3. Accéder à l'option Serverless

- Cliquez sur **Try Redshift Serverless free trial** si vous n'avez pas encore configuré de service Redshift Serverless.

3. Accéder à l'option Serverless

- Cliquez sur **Try Redshift Serverless free trial** si vous n'avez pas encore configuré de service Redshift Serverless.

4. Créer un Namespace (Espace de noms)

- Choisir **Customize Setting** et pas le default
- **Nom de l'espace de noms (Namespace Name)** : Donnez un nom à votre espace de noms (par exemple, `my-redshift-namespace`).
- **Base de données (Database)** : Choisissez un nom de base de données (par défaut `dev`, mais vous pouvez le personnaliser).
- **Identifiants d'accès** :
 - **Nom d'utilisateur administrateur** : Entrez un nom d'utilisateur administrateur, par exemple, `admin`.
 - **Mot de passe** : Créez un mot de passe sécurisé.

5. Créer un Workgroup (groupe de travail)

- **Nom du groupe de travail (Workgroup Name)** : Entrez un nom unique pour votre workgroup, par exemple, `my-serverless-workgroup`.
- **Limite maximale de RPU** : Spécifiez un maximum de **32 RPU** pour cette instance afin d'optimiser les coûts pour un petit projet.
- Redshift Serverless ajustera automatiquement la capacité en fonction de la demande, mais en limitant à 32 RPU, vous assurez des coûts maîtrisés.

6. Configuration sécurité :

- **VPC et sous-réseaux (subnet group)** : Sélectionnez le VPC dans lequel vous souhaitez exécuter Redshift Serverless. Si vous n'avez pas de sous-réseau disponible, AWS en créera un par défaut.
- **Paramètres de sécurité (Security settings)** :
 - Configurez les groupes de sécurité pour contrôler l'accès réseau. Vous pouvez laisser par défaut nous modifierons après la création du cluster.
 - Laissez les subnets par défauts créés par AWS

7. Examiner et créer

- Passez en revue toutes vos configurations (workgroup, namespace, VPC, utilisateurs, etc.).
- Cliquez sur **Créer** (Create) pour lancer l'instance Redshift Serverless.

8. Dashboard Redshift

Le tableau de bord Redshift Serverless fournit une vue d'ensemble des principaux composants et métriques de votre instance. Voici une description des sections clés que vous y trouverez :

1. Namespace Overview

Le **namespace** représente un espace logique dans Redshift Serverless où sont stockées les configurations et les métadonnées de vos bases de données. Il regroupe les informations liées aux bases de données, comme les utilisateurs, les schémas, et les paramètres de sauvegarde.

2. Namespace & Workgroups

- **Workgroup** : Le workgroup représente l'unité de calcul serverless. Il gère les ressources de traitement des requêtes et est associé à un namespace. Chaque workgroup peut s'adapter dynamiquement aux besoins en termes de performance en allouant un nombre variable de RPU (Redshift Processing Units).
- Un workgroup est lié à un **namespace** pour former un environnement complet pour l'exécution de vos charges de travail.
- Vous devriez normalement voir le workgroup que vous avez créé précédemment

3. Queries Metrics

Cette section montre les **métriques de requêtes** exécutées sur votre instance, notamment :

- Nombre de requêtes
 - Temps d'exécution
 - Taux de réussite ou d'échec des requêtes
- Ces informations vous aident à évaluer les performances des requêtes SQL exécutées.

4. Free Trial Information

Si vous êtes dans votre période d'essai gratuite, cette section indique combien d'heures ou de RPUs gratuits il vous reste avant de commencer à être facturé. Amazon Redshift Serverless offre souvent un quota gratuit de ressources pour les nouveaux utilisateurs.

5. Query Data

Cet onglet vous permet d'explorer et d'exécuter des requêtes SQL directement sur vos bases de données. Vous pouvez utiliser l'éditeur de requêtes intégré pour interagir avec vos tables et obtenir des résultats en temps réel.

6. Alarms

La section **Alarms** vous notifie des alertes de performance, de disponibilité ou d'usage anormal de votre instance. Vous pouvez configurer des alarmes via le service AWS **CloudWatch** pour être averti en cas de dépassement de seuils critiques.

7. RPU Capacity Used

Cette section affiche le **taux de consommation de RPUs** par votre workgroup. Vous y verrez combien d'unités de traitement sont actuellement allouées à vos requêtes et si votre utilisation se rapproche des limites configurées.

8. Feature Spotlight

Dans la section **Feature Spotlight**, AWS met en avant certaines fonctionnalités ou bonnes pratiques. Cela peut inclure des recommandations pour optimiser vos requêtes, tirer parti de nouvelles fonctionnalités ou améliorer les performances de votre base de données.

Paramétrage des identifiants RedShift avec AWS Secrets Manager

mardi 8 juillet 2025 10:35

Paramétrage des identifiants avec AWS Secrets Manager

Pourquoi utiliser AWS Secrets Manager ?

AWS Secrets Manager vous permet de stocker et de gérer vos informations sensibles (comme les noms d'utilisateur et mots de passe) en toute sécurité, au lieu de les écrire directement dans le code ou les systèmes de gestion de configurations. Cela permet :

- **Sécurité accrue** : Vos informations sensibles ne sont pas exposées dans le code.
- **Automatisation** : Secrets Manager permet une récupération automatique et simplifiée des informations sensibles dans vos applications.
- **Rotation automatique des secrets** : AWS peut régulièrement mettre à jour et changer vos secrets pour une meilleure sécurité.

Étape 1 : Accéder à AWS Secrets Manager

1. Connectez-vous à la **console de management AWS**.
2. Dans la **barre de recherche** en haut, tapez **Secrets Manager** et sélectionnez-le dans les résultats.

Étape 2 : Créer un nouveau secret

1. Dans Secrets Manager, cliquez sur **Store a new secret** (Créer un nouveau secret).
2. Sélectionnez **Credentials for Amazon Redshift** pour stocker des informations personnalisées comme vos **credentials Redshift**.
3. Sous **Key/value pairs** (Paires clé/valeur), ajoutez vos informations :
 - **Key (Clé)** : `username`
 - **Value (Valeur)** : votre nom d'utilisateur (ex : `admin`).
 - **Key (Clé)** : `password`
 - **Value (Valeur)** : votre mot de passe.
4. Choisissez le data warehouse auquel vous souhaitez associer le secret. Si vous avez correctement suivi les étapes précédentes, vous devriez voir votre workgroup Redshift Serverless dans la liste. Sélectionnez-le.
5. Cliquez sur **Next** (Suivant).

Étape 3 : Nommer et configurer le secret

1. **Nom du secret** : Donnez un nom descriptif à votre secret, par exemple `redshift/credentials`.
2. **Tags** (Optionnel) : Vous pouvez ajouter des balises pour organiser vos secrets. Pour une intégration avec Redshift, vous pouvez utiliser les tags suivants :
 - **Key (Clé)** : `Redshift`
 - **Value (Valeur)** : `default`
 - Vous pouvez ajuster la valeur selon vos besoins, mais utiliser `default` est une bonne pratique pour des identifiants généraux.
3. Cliquez sur **Next**.

Étape 4 : Configurer la rotation (optionnel)

- Si vous souhaitez que AWS gère la rotation automatique des secrets, activez l'option de rotation automatique et configurez les paramètres. Sinon, vous pouvez laisser cette option désactivée. Je l'ai laissé désactivé pour l'exercice.

Étape 5 : Révision et création

1. Vérifiez que toutes les informations sont correctes.
2. Cliquez sur **Store** (Enregistrer) pour créer le secret.

Étape 6 : Récupérer le secret dans un script Python

Maintenant que vos informations d'identification sont stockées dans **Secrets Manager**, vous pouvez les récupérer dans votre script Python. Vous pouvez récupérer votre secret à l'aide de python grâce au code qui vous ai fourni en bas de la page de aws secret. Le code ressemble à ceci :

```
1 # Use this code snippet in your app.
2 # If you need more information about configurations
3 # or implementing the sample code, visit the AWS docs:
4 # https://aws.amazon.com/developer/language/python/
5
6 import boto3
7 from botocore.exceptions import ClientError
8
9
10 def get_secret():
11
12     secret_name = "red_romain"
13     region_name = "eu-west-3"
14
15     # Create a Secrets Manager client
16     session = boto3.session.Session()
17     client = session.client(
18         service_name='secretsmanager',
19         region_name=region_name
20     )
21
22     try:
23         get_secret_value_response = client.get_secret_value(
24             SecretId=secret_name
25         )
26     except ClientError as e:
27         # For a list of exceptions thrown, see
28         # https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
29         raise e
30
31     secret = get_secret_value_response['SecretString']
32
```

1. Accéder à la Console de Gestion AWS

1. Connectez-vous à la **console de gestion AWS**.
2. Dans la barre de recherche, tapez **Amazon Redshift** et sélectionnez-le pour accéder à la gestion des instances Redshift.

2. Accéder aux Groupes de Sécurité via Redshift Serverless

1. Sélectionnez le workgroup (groupe de travail) associé à votre instance Redshift Serverless.
2. Déplacez vous sur la partie **Network and Security** (Réseau et sécurité) pour accéder aux paramètres réseau du workgroup.

3. Modifier le Groupe de Sécurité

1. Sous la section **VPC Security Group** (Groupes de sécurité VPC), vous verrez le ou les groupes de sécurité associés à votre workgroup Redshift Serverless.
2. Cliquez sur le lien du groupe de sécurité pour ouvrir ses détails : **le lien sera écrit sous la forme sg-xxxxxxxxxx**. Cela vous dirigera vers la configuration du groupe de sécurité directement depuis la console Redshift.

4. Ajouter une Règle d'Entrée (Inbound Rule)

1. Dans les détails du groupe de sécurité, allez à l'onglet **Inbound rules** (Règles entrantes).
2. Cliquez sur **Edit inbound rules** (Modifier les règles entrantes).
3. Cliquez sur **Add rule** (Ajouter une règle).
4. Configurez la nouvelle règle comme suit :
 - **Type** : Sélectionnez **Custom TCP Rule**.
 - **Protocol** : Le protocole sera automatiquement défini sur **TCP**.
 - **Port range** : Entrez le port de Redshift (par défaut, c'est **5439**).
 - **Source** : Sélectionnez **My IP** pour autoriser les connexions depuis votre adresse IP actuelle. Cela ajoutera automatiquement votre adresse IP à la règle, vous permettant d'accéder à Redshift depuis votre machine.
5. Cliquez sur **Save rules** (Enregistrer les règles) pour appliquer les modifications.

5. Configurer l'Accessibilité Publique (Publicly Accessible)

1. Retournez dans la console **Amazon Redshift**.
2. Sélectionnez votre cluster Redshift Serverless dans la liste des clusters.
3. Cliquez sur l'onglet **Properties** (Propriétés) pour accéder aux paramètres de votre cluster.
4. Dans la section **Network and Security** (Réseau et sécurité), trouvez l'option **Publicly accessible**.
5. Modifiez le paramètre **Publicly accessible** en **Yes** (Oui). Cette option permet à votre cluster Redshift d'être accessible depuis l'internet public, ce qui est nécessaire si vous devez vous connecter à Redshift depuis des endroits autres que les réseaux internes AWS.

Pourquoi ces configurations sont importantes

- **Configuration des Règles d'Entrée (Inbound Rules)** : En ajoutant une règle pour autoriser le trafic TCP sur le port de Redshift depuis votre adresse IP, vous vous assurez que vous pouvez accéder à votre cluster Redshift pour exécuter des requêtes et gérer vos données. Limiter l'accès à votre IP améliore la sécurité en restreignant les connexions uniquement à votre machine.
- **Option Publicly Accessible** : En activant l'option **Publicly accessible**, vous permettez à votre cluster Redshift d'être accessible via l'internet. Cela est nécessaire si vous souhaitez vous connecter à votre cluster depuis des machines qui ne sont pas sur le même réseau privé AWS.

Chargement et test de données avec SQLAlchemy

Avant d'exécuter le script Python pour importer des données dans Redshift, vous devez préparer votre environnement de développement. Suivez les étapes ci-dessous pour configurer votre environnement et installer les librairies nécessaires.

1. Préparer l'Environnement de Développement

1. **Ouvrir VS Code** : Lancez votre éditeur de code Visual Studio Code (VS Code).

2. **Créer un Environnement Virtuel** :

- Ouvrez un terminal dans VS Code.
- Créez un environnement virtuel en utilisant la commande suivante :

```
1 python -m venv venv
```

- Activez l'environnement virtuel :

- **Sur Windows** :

```
1 .\venv\Scripts\activate
```

- **Sur macOS/Linux** :

```
1 source venv/bin/activate
```

3. **Installer les Librairies Nécessaires** :

- Utilisez `pip` pour installer les librairies nécessaires au script. Exécutez les commandes suivantes dans le terminal :

```
1 pip install boto3 pandas==2.1.4 sqlalchemy-redshift psycopg2-binary
```

2. Explication du Code pour Importer des Données dans Redshift

Le script Python utilise AWS Secrets Manager pour récupérer les informations d'identification et se connecter à une base de données Redshift. Ensuite, il importe des données depuis un DataFrame Pandas dans Redshift. Voici une explication morceau par morceau du code.

2.1. Importation des Bibliothèques

```
1 import boto3
2 import json
3 import pandas as pd
4 from sqlalchemy import create_engine
5 from botocore.exceptions import ClientError
```

Explication du code :

- `import boto3`: Importe la bibliothèque AWS SDK pour Python, utilisée pour interagir avec AWS Secrets Manager.
- `import json`: Importe le module pour manipuler des données JSON, utilisé pour traiter les secrets récupérés.
- `import pandas as pd`: Importe la bibliothèque pour la manipulation des données, utilisée ici pour créer et gérer un DataFrame.
- `from sqlalchemy import create_engine`: Importe la fonction pour créer une connexion SQLAlchemy à la base de données.
- `from botocore.exceptions import ClientError`: Importe les exceptions spécifiques liées à boto3 pour la gestion des erreurs.

2.2 Fonction pour récupérer les secrets

```
1 def get_secret() -> dict[str, str]:
2     secret_name = "red_roman"
3     region_name = "eu-west-3"
4
5     # Create a Secrets Manager client
6     session = boto3.session.Session()
7     client = session.client(service_name="secretsmanager", region_name=region_name)
8
9     try:
10         get_secret_value_response = client.get_secret_value(SecretId=secret_name)
11     except ClientError as e:
12         # For a list of exceptions thrown, see
13         # https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
14         raise e
15
16     secret = get_secret_value_response["SecretString"]
17     secret_dict = json.loads(secret)
18
19     return secret_dict
```

Explication du code :

- `secret_name = "red_roman"`: Définit le nom du secret dans AWS Secrets Manager.
- `region_name = "eu-west-3"`: Spécifie la région AWS où le secret est stocké.
- `session = boto3.session.Session()`: Crée une session boto3 pour interagir avec AWS.
- `client = session.client(service_name="secretsmanager", region_name=region_name)`: Crée un client pour AWS Secrets Manager.
- `get_secret_value_response = client.get_secret_value(SecretId=secret_name)`: Récupère la valeur du secret à l'aide de son nom.
- `raise e`: Lève une exception en cas d'erreur lors de la récupération du secret.
- `secret_dict = json.loads(secret)`: Décode la chaîne JSON du secret en un dictionnaire Python.
- `return secret_dict`: Retourne le dictionnaire contenant les informations du secret.

2.3 Bloc Principal pour Connexion et Importation des Données

```
1 if __name__ == "__main__":
2     secret = get_secret()
3     username = secret.get("username")
4     password = secret.get("password")
5     host = secret.get("host")
6     database = "dev"
7     port = secret.get("port")
8
9     engine = create_engine(
10         f"redshift+psycopg2://{username}:{password}@{host}:{port}/{database}"
11     )
12
13     data = {"id": [1, 2, 3], "name": ["Alice", "Bob", "Charlie"], "age": [25, 30, 35]}
14     df = pd.DataFrame(data)
15
16     df.to_sql("test_table", engine, if_exists="replace", index=False, schema="public")
```

Explication du code :

- `secret = get_secret()`: Récupère les informations d'identification depuis AWS Secrets Manager.
- `username = secret.get("username")`: Récupère le nom d'utilisateur du secret.
- `password = secret.get("password")`: Récupère le mot de passe du secret.
- `host = secret.get("host")`: Récupère l'hôte de la base de données.
- `database = "dev"`: Spécifie le nom de la base de données à utiliser.
- `port = secret.get("port")`: Récupère le port de la base de données.
- `engine = create_engine(f"redshift+psycopg2://{username}:{password}@{host}:{port}/{database}")`: Crée une connexion SQLAlchemy à Redshift en utilisant les informations d'identification récupérées.
- `data = {"id": [1, 2, 3], "name": ["Alice", "Bob", "Charlie"], "age": [25, 30, 35]}`: Définit des données d'exemple à importer.
- `df = pd.DataFrame(data)`: Crée un DataFrame Pandas à partir des données d'exemple.
- `df.to_sql("test_table", engine, if_exists="replace", index=False, schema="public")`: Importe les données du DataFrame dans une table Redshift nommée test_table. Le paramètre `if_exists="replace"` remplace la table si elle existe déjà, et `index=False` évite d'importer les index du DataFrame.

Voici le code complet du script pour importer des données dans Redshift

```
1 import boto3
2 import json
3 import pandas as pd
4 from sqlalchemy import create_engine
5 from botocore.exceptions import ClientError
6
7 def get_secret() -> dict(str, str):
8     secret_name = "red_roman"
9     region_name = "eu-west-3"
10
11     # Create a Secrets Manager client
12     session = boto3.session.Session()
13     client = session.client(service_name="secretsmanager", region_name=region_name)
14
15     try:
16         get_secret_value_response = client.get_secret_value(SecretId=secret_name)
17     except ClientError as e:
18         # For a list of exceptions thrown, see
19         # https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
20         raise e
21
22     secret = get_secret_value_response["SecretString"]
23     secret_dict = json.loads(secret)
24
25     return secret_dict
26
27 if __name__ == "__main__":
28     secret = get_secret()
29     username = secret.get("username")
30     password = secret.get("password")
31     host = secret.get("host")
32     database = "dev"
33     port = secret.get("port")
34
35     engine = create_engine(
36         f"redshift+psycopg2://{username}:{password}@{host}:{port}/{database}"
37     )
38
39     data = {"id": [1, 2, 3], "name": ["Alice", "Bob", "Charlie"], "age": [25, 30, 35]}
40     df = pd.DataFrame(data)
41
42     df.to_sql("test_table", engine, if_exists="replace", index=False, schema="public")
```

```
import boto3
import json
import pandas as pd
from sqlalchemy import create_engine
from botocore.exceptions import ClientError
def get_secret() -> dict(str, str):
    secret_name = "red_roman"
    region_name = "eu-west-3"
    # Create a Secrets Manager client
    session = boto3.session.Session()
    client = session.client(service_name="secretsmanager",
        region_name=region_name)
    try:
        get_secret_value_response =
client.get_secret_value(SecretId=secret_name)
    except ClientError as e:
        # For a list of exceptions thrown, see
        #
https://docs.aws.amazon.com/secretsmanager/latest/apireference/API\_GetSecretValue.html
        raise e
    secret = get_secret_value_response["SecretString"]
    secret_dict = json.loads(secret)
    return secret_dict
if __name__ == "__main__":
    secret = get_secret()
    username = secret.get("username")
    password = secret.get("password")
    host = secret.get("host")
    database = "dev"
    port = secret.get("port")
    engine = create_engine(
        f"redshift+psycopg2://{username}:{password}@{host}:{port}/{database}"
    )
    data = {"id": [1, 2, 3], "name": ["Alice", "Bob", "Charlie"], "age": [25,
30, 35]}
    df = pd.DataFrame(data)
    df.to_sql("test_table", engine, if_exists="replace", index=False,
        schema="public")
```

Vérification de l'Importation des Données dans Redshift

Après avoir exécuté le script Python pour importer des données dans Redshift, suivez les étapes ci-dessous pour vérifier que les données ont été correctement importées et que la table est bien présente dans votre base de données.

1. Accéder à la Console Redshift

1. Ouvrir la Console Redshift :
 - Connectez-vous à votre console AWS.
 - Accédez au service **Amazon Redshift**.
2. Naviguer vers le Dashboard Redshift :
 - Sélectionnez le cluster Redshift approprié ou le Workgroup Redshift Serverless que vous utilisez.

2. Se Connecter à la Base de Données

1. Accéder à Query Data :
 - Dans le menu de gauche, cliquez sur **Query Data**.
2. Se Connecter à la Base de Données avec Secrets Manager :
 - Utilisez les informations d'identification stockées dans AWS Secrets Manager pour vous connecter à la base de données. Assurez-vous d'utiliser les mêmes paramètres (hôte, port, utilisateur, mot de passe) que ceux définis dans votre script Python.

3. Vérifier la Présence de la Table

1. Exécuter une Requête pour Vérifier la Table :

- Exécutez une requête SQL pour vérifier que la table `test_table` est présente dans la base de données. Utilisez la commande suivante :

```
1 SELECT *
2 FROM information_schema.tables
3 WHERE table_name = 'test_table';
```

2. Vérifier la Structure de la Table :

- Pour vérifier la structure de la table, vous pouvez exécuter la commande suivante :

```
1 SELECT column_name, data_type
2 FROM information_schema.columns
3 WHERE table_name = 'test_table';
```

4. Vérifier les Données Importées

1. Exécuter une Requête SELECT pour Visualiser les Données :

- Exécutez une requête SQL pour afficher les données de la table `test_table` et vérifier que les données importées correspondent au DataFrame Pandas utilisé dans le script. Utilisez la commande suivante :

```
1 SELECT *
2 FROM test_table;
```

Nettoyage des Ressources AWS

Pour éviter des frais inutiles après avoir terminé vos tâches, assurez-vous de nettoyer toutes les ressources AWS utilisées. Voici les étapes à suivre pour effectuer le nettoyage via la ligne de commande AWS CLI, ainsi que les manipulations nécessaires via la console AWS Management.

1. Supprimer le Workgroup Redshift Serverless

Via AWS CLI :

Exécutez la commande suivante pour supprimer le workgroup :

```
1 aws redshift-serverless delete-workgroup --workgroup-name <votre_workgroup_name>
```

Via la Console AWS Management :

- 1.Connectez-vous à la Console AWS.
- 2.Accédez à Redshift Serverless dans la section Services.
- 3.Dans le panneau de navigation, cliquez sur Workgroups.
- 4.Sélectionnez le workgroup que vous souhaitez supprimer.
- 5.Cliquez sur le bouton Actions et choisissez Delete.
- 6.Confirmez la suppression.

2. Supprimer le Namespace Redshift Serverless

Via AWS CLI :

Utilisez la commande suivante pour effacer le namespace :

```
1 aws redshift-serverless delete-namespace --namespace-name <votre_namespace_name>
```

Via la Console AWS Management:

- 1.Connectez-vous à la Console AWS.
- 2.Accédez à Redshift Serverless dans la section Services.
- 3.Dans le panneau de navigation, cliquez sur Namespaces.
- 4.Sélectionnez le namespace que vous souhaitez supprimer.
- 5.Cliquez sur le bouton Actions et choisissez Delete.
- 6.Confirmez la suppression.

3. Supprimer le Secret dans AWS Secrets Manager

Pour vérifier et supprimer le secret, utilisez les commandes suivantes :

```
1 aws secretsmanager list-secrets --query 'SecretList[*].ARN' --output text
```

Cela affichera la liste des secrets. Pour supprimer un secret spécifique, utilisez :

```
1 aws secretsmanager delete-secret --secret-id <votre_secret_id>
```

Via la Console AWS Management:

- 1.Connectez-vous à la Console AWS.
- 2.Accédez à Secrets Manager dans la section Services.
- 3.Trouvez le secret que vous souhaitez supprimer dans la liste.
- 4.Cliquez sur le nom du secret pour accéder aux détails.
- 5.Cliquez sur le bouton Delete.
- 6.Confirmez la suppression.

4. Vérifier les Clés dans AWS Key Management Service (KMS)

1. Connectez-vous à la Console AWS.
2. Accédez à Key Management Service (KMS) dans la section Services.
3. Dans le panneau de navigation, cliquez sur Keys.
4. Vérifiez si des clés sont actives. Pour chaque clé, vous pouvez :
 - Sélectionner la clé.
 - Cliquer sur Actions et choisir Disable pour désactiver une clé active.
 - Si nécessaire, vous pouvez également Delete la clé après l'avoir désactivée.

Introduction à AWS et à ses Services Clés

Présentation d'AWS

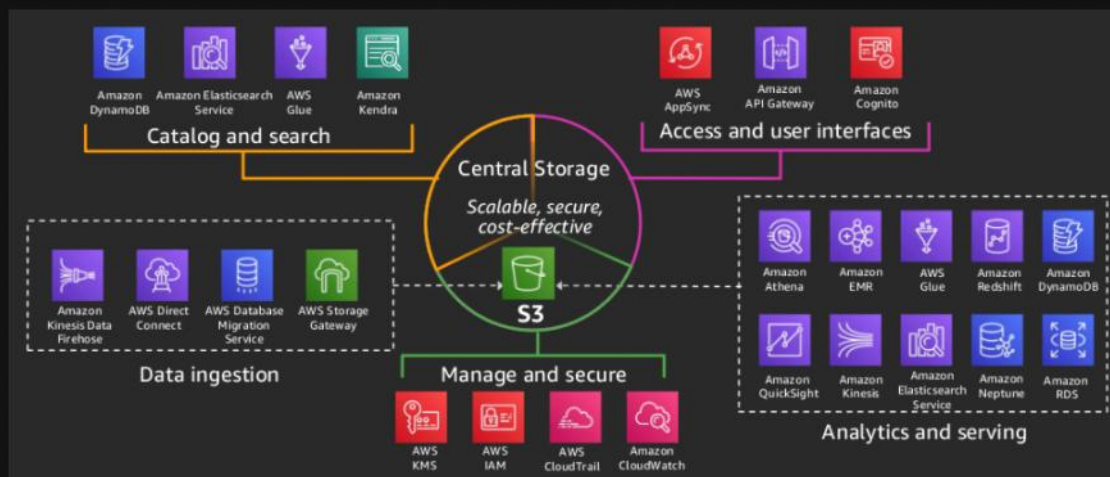
Amazon Web Services (AWS) est l'une des plateformes de cloud computing les plus populaires et les plus robustes disponibles aujourd'hui. Lancée par Amazon en 2006, AWS offre une vaste gamme de services cloud couvrant des domaines tels que le calcul, le stockage, les bases de données, l'analyse, l'intelligence artificielle, et bien plus encore. En tant que leader du marché, AWS fournit des solutions évolutives et flexibles adaptées à des besoins variés, des startups aux grandes entreprises.

Services Clés d'AWS

Amazon Web Services (AWS) propose une large gamme de services essentiels pour la gestion, l'analyse et l'exploitation des données à grande échelle. Voici quelques-uns des services clés d'AWS particulièrement pertinents pour les professionnels de la donnée :

- **Amazon EC2 (Elastic Compute Cloud)** : EC2 offre une capacité de calcul redimensionnable dans le cloud, ce qui permet de lancer des instances de serveurs virtuels en quelques minutes. Pour les projets data, EC2 est idéal pour exécuter des applications analytiques lourdes, des bases de données, ou des frameworks de traitement de données comme Apache Hadoop ou Spark.
- **Amazon S3 (Simple Storage Service)** : S3 est une solution de stockage d'objets évolutive et hautement durable, parfaite pour stocker de grandes quantités de données structurées et non structurées. C'est le service de choix pour les data lakes, les sauvegardes et l'archivage des données.
- **Amazon RDS (Relational Database Service)** : RDS simplifie la gestion des bases de données relationnelles en automatisant des tâches telles que les sauvegardes, les mises à jour logicielles, et la réplication. Il prend en charge plusieurs moteurs de base de données comme MySQL, PostgreSQL, MariaDB, Oracle et SQL Server, facilitant ainsi la gestion de bases de données critiques pour vos applications data-driven.
- **Amazon Redshift** : Redshift est un service de data warehouse rapide et entièrement géré, capable d'analyser des téraoctets, voire des pétaoctets de données à travers des requêtes SQL. C'est une solution optimale pour des analyses complexes, des rapports BI, et des dashboards analytiques.
- **AWS Glue** : Glue est un service ETL (Extract, Transform, Load) sans serveur qui simplifie l'intégration et la transformation des données provenant de diverses sources. Il est essentiel pour alimenter vos data lakes et data warehouses en données propres et structurées.
- **Amazon Athena** : Athena permet d'exécuter des requêtes SQL interactives directement sur les données stockées dans S3. Cela en fait un outil puissant pour explorer des datasets volumineux sans avoir à les déplacer dans un data warehouse.

Illustration des services orientés Data de AWS



Pourquoi AWS est-il un choix populaire ?

AWS est devenu un choix populaire pour les entreprises et les professionnels de la donnée pour plusieurs raisons :

- **Évolutivité et Flexibilité** : AWS offre une infrastructure hautement évolutive qui permet aux entreprises de s'adapter rapidement à la croissance de leurs besoins en données. Les services AWS peuvent être facilement redimensionnés pour gérer des volumes de données croissants ou pour répondre à des pics de charge sans compromettre les performances.
- **Large Écosystème de Services** : AWS propose un ensemble complet de services cloud qui couvre tous les aspects de la gestion et de l'analyse des données, de l'infrastructure de base (comme EC2) à des services avancés de machine learning, d'analytique et de data warehousing. Cela permet aux entreprises de construire des solutions complexes entièrement dans le cloud sans avoir à intégrer de services tiers.
- **Sécurité et Conformité** : AWS met l'accent sur la sécurité des données avec des fonctionnalités telles que le chiffrement des données en transit et au repos, des contrôles d'accès granulaires et une conformité avec de nombreuses normes internationales (comme GDPR, HIPAA, et SOC). Cela permet aux entreprises de protéger leurs données sensibles et de respecter les réglementations en vigueur.
- **Modèle de Tarification Pay-as-You-Go** : AWS adopte un modèle de tarification à l'usage, ce qui signifie que les entreprises ne paient que pour les ressources qu'elles consomment. Cela permet d'optimiser les coûts, surtout pour les startups et les petites entreprises qui peuvent ajuster leurs dépenses en fonction de leurs besoins réels.
- **Fiabilité et Haute Disponibilité** : AWS dispose d'un réseau mondial de centres de données redondants, garantissant une haute disponibilité des services et une résilience face aux pannes. Les entreprises peuvent compter sur AWS pour maintenir leurs applications et leurs données disponibles 24/7.
- **Communauté et Support** : Avec une large communauté d'utilisateurs, une documentation exhaustive, et des options de support technique variées, AWS offre les ressources nécessaires pour aider les entreprises à résoudre les problèmes rapidement et à tirer le meilleur parti de ses services.